

CANDY DIALOGUE ENGINE

Speech Bubbles Customization

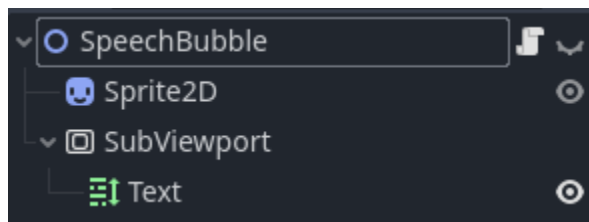
1.0.0

Speech bubbles are used by the "Bubbles" and "VN Bubbles" dialogue modes to display dialogue inside a comic-book style speech bubble, positioned near the speaking character's sprite or mesh.

Two scenes and scripts are provided: one pair for 2D speech bubbles, and another pair for 3D speech bubbles.

Speech bubbles are a special case: there isn't just one single way of displaying speech inside bubbles in Godot. You may decide that you'd rather use a different method entirely.

Default Speech Bubbles



The default provided speech bubbles scenes use a RichTextLabel to display spoken text. This RichTextLabel is displayed in a Sprite3D, through a ViewportDisplay node.

The RichTextLabel is assigned a StyleBoxTexture theme override, which displays a speech bubble texture. You may replace it with a texture of your own, but you may need to adjust the content margins to let the text fit correctly.

What this means:

You can create your own speech bubble setup, as long as it contains the required RichTextLabel and the required functions.

However, using a different node setup will require that you rewrite some of the code in [Speech_Bubble_2D.gd](#) and [Speech_Bubble_3D.gd](#) to make it compatible.

Don't worry: most users don't need to use a custom setup. If you need to, you'll know.

Limitations:

The method used by the default speech bubbles prevents the mouse from interacting with the BBCode [hint] and [url] tags. This prevents speech bubbles from displaying tooltips for Lexicon keywords, or making keywords clickable.

For these tags to work properly, the RichTextLabel must descend from a CanvasLayer (from our tests, the CanvasLayer doesn't need to be a direct parent).

However, using a CanvasLayer would cause the speech bubbles to always overlap the game environment, and prevent them from being hidden by obstacles, which may feel less immersive.

Nothing stops you from using your own speech bubbles that use the CanvasLayer method. Just be sure to make their code compatible.

Nodes

Root

The root node should be named "SpeechBubble", as this is the name the dialogue engine seeks in some places.

This is actually only required if you attach speech bubbles to character busts in Bust (VN) Scenes. However, to ensure your game is compatible with future updates, we recommend naming the root node "SpeechBubble" in every case.

Required Nodes

The dialogue engine requires the following nodes:

RichTextLabel

- Displays the spoken text (and optionally the speaker's name).
- Requires BBCode enabled.
- Requires Fit Content enabled.
- Recommend Autowrap Mode set to "Word" or "Word (Smart)".
- Script pointer: @onready var **label**

Optional Nodes

These nodes are required by the default **Speech_Bubble_2D.gd** and **Speech_Bubble_3D.gd** script, but are not required by the dialogue engine directly.

In other words, you do not need them if you use a different method of displaying speech bubbles.

ViewportDisplay

- Displays the RichTextLabel through the Sprite2D or Sprite3D node.
- Script pointer: @onready var **viewport**

Sprite2D / Sprite3D

- Displays the ViewportDisplay, and therefore the RichTextLabel
- Script pointer: @onready var **sprite**

Script

Base

The script **Speech_Bubble_2D.gd** or **Speech_Bubble_3D.gd** should be attached to the scene's root node, depending whether the root node is a 2D or 3D node.

Variables

offset_y / offset_x / offset_z

These variables manually control the speech bubble's vertical and horizontal offset in relation to the character node they are parented to.

Taller or smaller characters may require adjusting the Y offset.

The X and Z offset may be adjusted if you want speech bubbles to appear slightly to the side of the character, instead of being centered. Note that 2D speech bubbles don't have the **bubble_offset_z** variable.

All three variables are exported and can be modified in the Inspector, allowing you to give each character its own custom offset.

max_width

This variable represents the maximum width speech bubbles can stretch.

When displaying text, speech bubbles will first stretch up to that limit. Then, if the text is longer, it will be displayed over multiple lines.

This variable is exported and can be modified in the Inspector.

test_width_margin

This variable adds some additional width to the text node, to prevent text from touching the edges. This is purely cosmetic, but important for a polished look.

This variable is exported and can be modified in the Inspector.

override_default_color

The default color to use for spoken text in the dialogue box.

Overrides **default_bubble_font_color** (**Candy_Engine.gd**). Is overridden by "**Bubble Text Color**" in the **actors** dictionary (**Candy_Database.gd**)

This variable is exported and can be modified in the Inspector.

override_color

An internal variable, used by the dialogue engine to store the final color to use for text display.

default_z

The default z_index value for the speech bubble. Must be lower than the values of any video or image players.

Functions

ready()

Contains code to connect the "meta_clicked" signal, in the event that you use the Lexicon feature with clickable keywords. See the **Lexicon** guide for details.

show_text()

Called by the dialogue engine directly. This function assigns the text to the RichTextLabel node, ensures the RichTextLabel and the ViewportDisplay are resized correctly, and offsets the entire bubble as per your settings.

If your speech bubbles use a different method than the default bubbles, the actual code in show_text() doesn't matter. The function needs to be in the speech bubble's script to prevent a crash when the dialogue engine calls it, but the code can be replaced with any logic your bubbles require (at minimum, it should assign the spoken text to the RichTextLabel and set visible = true on the bubble's root node).

clear()

This function is called when speech bubbles need to be cleared from the UI, such as when the dialogue engine continues to a new Spoken Line, when the dialogue ends, or when the \$Clear command is used.

write_begun()

This is a hook function, called just before dialogue starts writing (after everything else about the spoken line has been calculated). The function is intended to be edited if you want to trigger custom behavior at that moment.

The function is called with the 'await' method.

write_finished()

This is a hook function, called when dialogue has finished writing. The function is intended to be edited if you want to trigger custom behavior at that moment.

The function is called with the 'await' method.

Recap

To prevent any errors, we suggest abiding by the following recommendations:

- Use the provided speech bubble scenes and scripts without modifying the node structure.
- Replace the texture in the RichTextLabel's Theme Overrides / Styles if you want to give your speech bubbles a custom look.
- Modify the exported variables as needed.
- Don't touch anything else, unless you have a good reason and you know what you're doing.
- Make backups of the default scenes and scripts, just in case.

To clarify, using your own speech bubbles is absolutely possible, but it can involve somewhat complex coding to work right, irrespective of Candy DE.

You'll also need to ensure that all the needed variables and functions are present in your script, or adapt the engine's code.